

Examen Práctico Complementario: Programación Python

Bitácora de Interacción con Inteligencia Artificial

Andrea Arroyo Escalante

Matrícula: 35163832

Materia: Computación Aplicada

Ciclo Escolar: Semestre Agosto-Diciembre 2026

Catedrático: Carlos Héctor Castañeda Ramírez

22 de septiembre de 2026

Documento de evidencia del proceso de interacción con herramientas de Inteligencia Artificial.

1. Introducción

Este documento pertenece a la parte práctica del Primer Examen Parcial de la materia Computación Aplicada del semestre Agosto-Diciembre 2026 de la Universidad Autónoma de Zacatecas. Aquí se muestra paso a paso la interacción que se tuvo con la inteligencia artificial de *Google Gemini* para resolver el problema práctico planteado en el examen que plantea *"Desarrollo de una aplicación de consola en Python (p085-simulador-venta-combustible.py) empleando ingeniería de prompts gradual con Inteligencia Artificial."*. Aquí se muestra la evidencia de las cuatro fases.

2. Planteamiento del problema

Se requiere desarrollar un sistema interactivo de consola para la gestión de una estación de servicio (gasolinera). El programa debe permitir a los operadores realizar cálculos de ventas, proyecciones de rendimiento y categorización de clientes mediante un flujo lógico robusto.

Funcionalidades del Sistema:

1. **Menú Principal:** El programa debe ejecutarse en un ciclo infinito (`while True`) que presente cuatro opciones:
 - Venta de Combustible.
 - Simulación de Rendimiento.
 - Clasificador de Cliente.
 - Salir.
2. **Módulo de Venta:** Solicitar el tipo de combustible, precio por litro y cantidad de litros. Debe validar que los valores sean positivos. El cálculo del total debe presentarse con f-strings, alineación a la derecha y dos decimales.
3. **Simulación de Rendimiento:** Utilizando un ciclo `for`, el usuario ingresará un kilometraje inicial y se debe mostrar una tabla proyectada (mes a mes o km a km) del consumo estimado basado en un factor de rendimiento constante proporcionado por el usuario.
4. **Clasificador de Cliente:** Mediante estructuras `if/elif/else` y operadores lógicos, determinar si un cliente es "Regular", "Premium" o "Flotilla" basándose en el volumen de compra mensual (ej. < 100 L, $100 - 500$ L, > 500 L).

5. **Finalización:** La opción de salida debe terminar el programa de forma limpia.

Requisitos Técnicos Obligatorios

- **Variables y Tipos:** Uso correcto de `int`, `float` y `str`.
- **Operadores Aritméticos:** Implementación obligatoria de división entera (`//`), residuo (`%`) y potencia (`**`) en cálculos de simulación o validación.
- **Formateo:** Uso de f-strings para todas las salidas de datos, cuidando la estética de la consola.
- **Control de Flujo:** Uso de `while` con sentencias `break` y `continue`, y `for` con la función `range()`.
- **Restricción Estricta:** Queda terminantemente prohibido el uso de listas, tuplas, diccionarios, funciones personalizadas o cualquier estructura de datos avanzada no cubierta en el Bloque 1.

3. Guía de Ejecución Gradual con IA

Orden en el que se interactuó con un modelo de lenguaje (LLM), en este caso fue *Google Gemini*, dividida en cuatro fases.

FASE 1: Descomposición y Diseño de Algoritmo

"Solicita a la IA que te ayude a estructurar la lógica antes de codificar."

A continuación se muestra la interacción con *Google Gemini* en forma de texto para la primera parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo A se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT: *"Actúa como un experto en Python. Ayúdame a diseñar el pseudocódigo para un menú interactivo que use un ciclo while y una estructura de control para una gasolinera, sin usar funciones ni listas. El menú debe presentar cuatro opciones: 1) venta de combustible para solicitar el tipo de combustible, precio por litro y cantidad de litros; 2) simulación de rendimiento para que a partir de un kilometraje inicial se muestre el consumo estimado mes a mes o km a km en forma de tabla utilizando un ciclo for basado en un factor de rendimiento constante proporcionado por el usuario; 3) clasificador de cliente para determinar si un cliente es "Regular", "Premium" o "Flotilla" basándose en el*

volumen de compra mensual mediante estructuras if/elif/else y operadores lógicos; y 4) salir para terminar el programa de forma limpia.”

RESPUESTA:

```
Plaintext
// Inicialización de la variable de control del ciclo
opcion = 0

MIENTRAS opcion != 4 HACER:
    // -----
    // DESPLIEGUE DEL MENÚ INTERACTIVO
    // -----
    IMPRIMIR "\n=====
    IMPRIMIR "      SISTEMA DE GESTIÓN DE GASOLINERA"
    IMPRIMIR "=====
    IMPRIMIR "1. Venta de combustible"
    IMPRIMIR "2. Simulación de rendimiento"
    IMPRIMIR "3. Clasificador de cliente"
    IMPRIMIR "4. Salir"
    IMPRIMIR "-----"

    LEER opcion

    // -----
    // ESTRUCTURA DE CONTROL PARA LAS OPCIONES DEL MENÚ
    // -----
    SI opcion == 1 ENTONCES:
        IMPRIMIR "\n--- REGISTRO DE VENTA DE COMBUSTIBLE ---"
        IMPRIMIR "Ingrese tipo de combustible (Magna / Premium / Diesel):"
        LEER tipo_combustible

        IMPRIMIR "Ingrese precio por litro:"
        LEER precio_litro

        IMPRIMIR "Ingrese cantidad de litros comprados:"
        LEER cantidad_litros

        // Cálculo del costo total
        total_pagar = precio_litro * cantidad_litros

        IMPRIMIR "\n--- RESUMEN DE COMPRA ---"
        IMPRIMIR "Combustible: " + tipo_combustible
        IMPRIMIR "Litros: " + cantidad_litros + " L"
        IMPRIMIR "Total a pagar: $" + total_pagar

    SINO SI opcion == 2 ENTONCES:
        IMPRIMIR "\n--- SIMULACIÓN DE RENDIMIENTO MES A MES ---"
        IMPRIMIR "Ingrese el kilometraje inicial del vehículo:"
        LEER km_inicial

        IMPRIMIR "Ingrese los kilómetros promedio recorridos por mes:"
        LEER km_por_mes

        IMPRIMIR "Ingrese el número de meses a simular:"
        LEER total_meses

        IMPRIMIR "Ingrese el factor de rendimiento constante (km por litro):"
        LEER factor_rendimiento
```

```

// Encabezado de la tabla
IMPRIMIR "\nMes | Km Inicial | Km Final | Consumo Est. (L)"
IMPRIMIR "-----"

km_actual = km_inicial

// Ciclo FOR para la proyección paso a paso
PARA mes DESDE 1 HASTA total_meses HACER:
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento

    IMPRIMIR mes + " | " + km_actual + " km | " + km_final + " km | " + consumo_mes + " L"

// Actualización del kilometraje acumulado para la siguiente iteración
km_actual = km_final

SINO SI opcion == 3 ENTONCES:
    IMPRIMIR "\n--- CLASIFICACIÓN DE CLIENTES ---"
    IMPRIMIR "Ingrese el volumen de compra mensual en litros:"
    LEER litros_mes

    IMPRIMIR "¿El cliente es una empresa registrada? (SI / NO):"
    LEER es_empresa

    // Clasificación mediante IF / ELIF / ELSE y operadores lógicos (AND, OR)
    SI litros_mes >= 1000 O (es_empresa == "SI" Y litros_mes >= 500) ENTONCES:
        categoria = "Flotilla"
        descuento = "10%"
    SINO SI litros_mes >= 300 Y litros_mes < 1000 ENTONCES:
        categoria = "Premium"
        descuento = "5%"
    SINO:
        categoria = "Regular"
        descuento = "0%"

    IMPRIMIR "\n--- RESULTADO DE CATEGORIZACIÓN ---"
    IMPRIMIR "Categoría asignada: " + categoria
    IMPRIMIR "Descuento aplicable: " + descuento

SINO SI opcion == 4 ENTONCES:
    IMPRIMIR "\nSaliendo del sistema... ¡Gracias por su visita!"

SINO:
    IMPRIMIR "\n[ERROR] Opción no válida. Por favor, seleccione un número entre 1 y 4."

```

Elementos clave del diseño:

- *Ciclo principal while*: Mantiene el programa activo continuamente manipulando la variable de control `opcion` hasta que se selecciona la opción de salida (4).
- *Estructura selectiva if / elif / else*: Rutea el flujo hacia la tarea seleccionada y valida que el usuario introduzca una opción válida en la entrada.
- *Ciclo for independiente*: Utilizado en la opción 2 para iterar un rango fijo de meses calculando y formateando incrementalmente las lecturas de kilometraje y consumo en forma de tabla sin usar listas.
- *Operadores lógicos (AND/OR)*: Implementados en la opción 3 para evaluar múltiples

condiciones de volumen de litros e identificar clientes comerciales para la categoría "Flotilla".

Observaciones

Escribir lo que queremos hacer en forma de pseudocódigo nos sirve para estructurar la lógica de programa a crear antes de que nos pongamos a escribir el código, nos ayuda a entender el problema y organizar una posible solución para saber de dónde empezar.

FASE 2: Implementación Incremental

"Desarrolla los módulos de cálculo uno por uno."

A continuación se muestra las interacciones con *Google Gemini* en forma de texto para la segunda parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo B se muestra la evidencia de la interacción en forma de capturas de pantalla.

MÓDULO 1: Venta de combustible

PROMPT: *"Para el primer módulo, escribe el código en Python para calcular el total de una venta de combustible validando que los datos sean float positivos. Usa f-strings para mostrar el resultado con 2 decimales y el símbolo de pesos alineado."*

RESPUESTA:

```
Python
print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")

tipo_combustible = input("Ingrese el tipo de combustible (ej. Magna, Premium, Diesel): ")

# Validación del precio por litro (debe ser float positivo)
precio_litro = 0.0
while precio_litro <= 0:
    try:
        precio_litro = float(input("Ingrese el precio por litro: "))
        if precio_litro <= 0:
            print("[ERROR] El precio debe ser un valor positivo mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Validación de la cantidad de litros (debe ser float positivo)
cantidad_litros = 0.0
while cantidad_litros <= 0:
    try:
        cantidad_litros = float(input("Ingrese la cantidad de litros: "))
        if cantidad_litros <= 0:
            print("[ERROR] La cantidad de litros debe ser mayor a cero.")
    except ValueError:
```

```

    print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Cálculo del importe total
total_pagar = precio_litro * cantidad_litros

# Despliegue del ticket con alineación y 2 decimales usando f-strings
print("\n" + "=" * 40)
print("          TICKET DE VENTA          ")
print("=" * 40)
print(f"Combustible:          {tipo_combustible}")
print(f"Precio por litro: $ {precio_litro:>10.2f}")
print(f"Litros cargados:      {cantidad_litros:>10.2f} L")
print("-" * 40)
print(f"Total a pagar:        $ {total_pagar:>10.2f}")
print("=" * 40)

```

Detalles implementación:

- *Validación robusta:* Los ciclos `while` junto con bloques `try/except ValueError` garantizan que el programa no colapse si el usuario ingresa texto y no avanzan hasta recibir un número float estrictamente mayor a 0.
- *Formato de alineación:* La sintaxis `>10.2f` dentro del *f-string* reserva un ancho de 10 caracteres alineados a la derecha y restringe la salida a exactamente 2 decimales, manteniendo la alineación del signo de pesos \$.

MÓDULO 2: Simulación de rendimiento

PROMPT: *"Para el segundo módulo, escribe el código en Python para calcular el consumo estimado de un vehículo y, utilizando un ciclo for, mostrarlo en una tabla proyectada (mes a mes o km a km) del consumo estimado basado en un factor de rendimiento constante. El usuario ingresará el kilometraje inicial y el factor de rendimiento."*

RESPUESTA:

```

Python
print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

# Validación de Kilometraje Inicial (float >= 0)
km_inicial = -1.0
while km_inicial < 0:
    try:
        km_inicial = float(input("Ingrese el kilometraje inicial del vehículo (km): "))
        if km_inicial < 0:
            print("[ERROR] El kilometraje inicial no puede ser negativo.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de Factor de Rendimiento en km/L (float > 0)
factor_rendimiento = 0.0
while factor_rendimiento <= 0:
    try:
        factor_rendimiento = float(input("Ingrese el factor de rendimiento (km por litro): "))
        if factor_rendimiento <= 0:

```

```

        print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de kilómetros recorridos por mes (float > 0)
km_por_mes = 0.0
while km_por_mes <= 0:
    try:
        km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes (km): "))
        if km_por_mes <= 0:
            print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación del número de meses para el ciclo for (int > 0)
total_meses = 0
while total_meses <= 0:
    try:
        total_meses = int(input("Ingrese la cantidad de meses a simular: "))
        if total_meses <= 0:
            print("[ERROR] El número de meses debe ser al menos 1.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

# Despliegue del encabezado de la tabla formateada
print("\n" + "=" * 62)
print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':^62}")
print("=" * 62)
print(f"{'Mes':^6} | {'Km Inicial':^12} | {'Km Final':^12} | {'Consumo Est. (L)':^18}")
print("-" * 62)

km_actual = km_inicial
consumo_acumulado = 0.0

# Ciclo FOR para calcular e imprimir paso a paso sin utilizar listas
for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento
    consumo_acumulado += consumo_mes

    # Impresión con alineación y decimales usando f-strings
    print(f"{mes:^6d} | {km_actual:>12.1f} | {km_final:>12.1f} | {consumo_mes:>18.2f}")

    # Actualización de variable para la siguiente iteración
    km_actual = km_final

print("=" * 62)
print(f"Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
print(f"Consumo total acumulado: {consumo_acumulado:>10.2f} L")
print("=" * 62)

```

Detalles implementación:

- *Generación de la tabla:* Utiliza el rango estático `range(1, total_meses + 1)` dentro del ciclo `for` para avanzar mes a mes sin necesidad de listas ni arreglos.
- *Alineación con f-strings:*
 - `:^6`: Centra el número de mes en un espacio de 6 caracteres.

- `>12.1f`: Alinea los kilometrajes a la derecha reservando 12 espacios con 1 decimal.
- `>18.2f`: Alinea los litros proyectados a la derecha reservando 18 espacios con 2 decimales.
- *Control de variables acumuladoras*: Actualiza el estado de `km_actual` y `consumo_acumulado` en cada iteración manteniendo el cálculo continuo del odómetro.

MÓDULO 3: Clasificador de cliente

PROMPT: *"Para el tercer módulo, escribe el código en Python para determinar si un cliente es "Regular", "Premium" o "Flotilla", basándose en el volumen de compra mensual (ej. <100L, 100-500L, >500L), mediante estructuras if/elif/else y operadores lógicos."*

RESPUESTA:

```
Python
print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

# Validación del volumen de compra mensual (float >= 0)
volumen_mensual = -1.0
while volumen_mensual < 0:
    try:
        volumen_mensual = float(input("Ingrese el volumen de compra mensual en litros: "))
        if volumen_mensual < 0:
            print("[ERROR] El volumen de compra no puede ser un número negativo.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Solicitud de dato complementario para evaluar condiciones compuestas
es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()

# Clasificación del cliente mediante estructuras if/elif/else y operadores lógicos (and, or)
if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
    categoria = "Flotilla"
    descuento = 10.0
    beneficio = "Descuento preferencial + Facturación acumulada"
elif volumen_mensual >= 100 and volumen_mensual <= 500:
    categoria = "Premium"
    descuento = 5.0
    beneficio = "Descuento en consumos + Puntos dobles"
elif volumen_mensual >= 0 and volumen_mensual < 100:
    categoria = "Regular"
    descuento = 0.0
    beneficio = "Acumulación de puntos básicos"
else:
    categoria = "No clasificado"
    descuento = 0.0
    beneficio = "Sin beneficios"

# Despliegue de resultados con formato f-string
print("\n" + "=" * 52)
print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':^52}")
print("=" * 52)
```

```
print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
print(f"Registro corporativo:      {es_empresa:>12}")
print("-" * 52)
print(f"Categoría asignada:          {categoria}")
print(f"Descuento aplicable:         {descuento:>12.1f} %")
print(f"Beneficios del programa:     {beneficio}")
print("-" * 52)
```

Detalles de la solución:

- *Uso de operadores lógicos (and, or):*
 - Para Flotilla: Aplica si el volumen es estrictamente mayor a 500L O si es una empresa con compras desde 300L (`volumen_mensual >500 or (es_empresa == 'S' and volumen_mensual >= 300)`).
 - Para Premium: Evalúa que el volumen esté dentro del rango de 100L a 500L inclusive utilizando el operador and (`volumen_mensual >= 100 and volumen_mensual <= 500`).
 - Para Regular: Define compras estrictamente menores a 100L (`volumen_mensual >= 0 and volumen_mensual <100`).
- Tratamiento de cadenas: Limpia espacios con `.strip()` y convierte a mayúsculas con `.upper()` en la entrada del usuario para evitar errores de captura en la respuesta 'S'/'N'.

MÓDULO 4: Finalización

PROMPT: *"Para el cuarto módulo, escribe el código en Python para que termine el programa de forma limpia"*

RESPUESTA:

```
Python
print("\n" + "-" * 50)
print(f"{'CERRANDO SISTEMA DE GESTIÓN':~50}")
print("-" * 50)
print(" Guardando registros de la sesión...")
print(" Liberando memoria del sistema...")
print(" Estado de la terminal: Finalizada correctamente.")
print("-" * 50)
print(" ;Gracias por utilizar el sistema de la gasolinera!")
print(" ;Que tenga un excelente día!")
print("-" * 50)

# Cambio de la variable de control del ciclo while principal
# (Al cambiar el valor de la opción a 4 o 'ejecutando' a False,
# el ciclo finaliza de forma limpia sin romper abruptamente el programa)
opcion = 4
```

Detalles de la solución:

- *Terminación estructurada*:: A diferencia del uso de comandos de interrupción forzada como `sys.exit()`, el cierre se realiza mediante la modificación de la variable de control del ciclo `while` principal (`opcion = 4`). Esto permite que el flujo del programa complete su iteración de forma natural y limpia.
- **Confirmación visual**: Incluye un mensaje explícito de cierre que le da certeza al usuario de que las operaciones finalizaron correctamente antes de devolver el control a la terminal.

Observaciones

Con ayuda de *Google Gemini* fue posible empezar a crear el programa en lenguaje Python haciéndolo por partes, en módulos, para posteriormente juntarlos en la siguiente fase.

FASE 3: Integración y Flujo de Control

"Une las partes en el ciclo principal y asegura el uso de `break` y `continue`."

A continuación se muestran las interacciones con *Google Gemini* en forma de texto para la tercera parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo C se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT 1: *"Para el programa general, escribe el código en Python que muestre el menú y que junte los cuatro módulos, acomodalos de una forma adecuada e integra este código en un ciclo `while True`, si el usuario elige la opción 4, usa `break`, si el usuario ingresa una opción inválida, muestra un error y usa `continue` para reiniciar el menú."*

RESPUESTA 1:

```
Python
while True:
    # -----
    # DESPLIEGUE DEL MENÚ PRINCIPAL
    # -----
    print("\n" + "=" * 45)
    print("      SISTEMA DE GESTIÓN DE GASOLINERA")
    print("=" * 45)
    print("1. Venta de combustible")
    print("2. Simulación de rendimiento")
    print("3. Clasificador de cliente")
    print("4. Salir")
    print("=" * 45)

    opcion = input("Seleccione una opción (1-4): ").strip()

    # Validación de la opción seleccionada
    if opcion not in ("1", "2", "3", "4"):
        print("\n[ERROR] Opción no válida. Debe ingresar un número entre 1 y 4.")
```

```
        continue # Reinicia el ciclo while inmediatamente

# -----
# MÓDULO 1: VENTA DE COMBUSTIBLE
# -----
if opcion == "1":
    print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")
    tipo_combustible = input("Ingrese el tipo de combustible (ej. Magna, Premium, Diesel): ")

    # Validación del precio por litro
    precio_litro = 0.0
    while precio_litro <= 0:
        try:
            precio_litro = float(input("Ingrese el precio por litro: "))
            if precio_litro <= 0:
                print("[ERROR] El precio debe ser un valor positivo mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Validación de la cantidad de litros
    cantidad_litros = 0.0
    while cantidad_litros <= 0:
        try:
            cantidad_litros = float(input("Ingrese la cantidad de litros: "))
            if cantidad_litros <= 0:
                print("[ERROR] La cantidad de litros debe ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Cálculo del importe total
    total_pagar = precio_litro * cantidad_litros

    # Ticket de venta
    print("\n" + "=" * 40)
    print("          TICKET DE VENTA          ")
    print("=" * 40)
    print(f"Combustible:      {tipo_combustible}")
    print(f"Precio por litro: $ {precio_litro:>10.2f}")
    print(f"Litros cargados:   {cantidad_litros:>10.2f} L")
    print("-" * 40)
    print(f"Total a pagar:     $ {total_pagar:>10.2f}")
    print("=" * 40)

# -----
# MÓDULO 2: SIMULACIÓN DE RENDIMIENTO
# -----
elif opcion == "2":
    print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

    # Validación de Kilometraje Inicial
    km_inicial = -1.0
    while km_inicial < 0:
        try:
            km_inicial = float(input("Ingrese el kilometraje inicial del vehículo (km): "))
            if km_inicial < 0:
                print("[ERROR] El kilometraje inicial no puede ser negativo.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Validación de Factor de Rendimiento
    factor_rendimiento = 0.0
```

```

while factor_rendimiento <= 0:
    try:
        factor_rendimiento = float(input("Ingrese el factor de rendimiento (km por litro): "))
        if factor_rendimiento <= 0:
            print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de kilómetros mensuales
km_por_mes = 0.0
while km_por_mes <= 0:
    try:
        km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes (km): "))
        if km_por_mes <= 0:
            print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación del número de meses
total_meses = 0
while total_meses <= 0:
    try:
        total_meses = int(input("Ingrese la cantidad de meses a simular: "))
        if total_meses <= 0:
            print("[ERROR] El número de meses debe ser al menos 1.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

# Despliegue de la tabla proyectada
print("\n" + "=" * 62)
print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':~62}")
print("=" * 62)
print(f"{'Mes':~6} | {'Km Inicial':~12} | {'Km Final':~12} | {'Consumo Est. (L)':~18}")
print("-" * 62)

km_actual = km_inicial
consumo_acumulado = 0.0

for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento
    consumo_acumulado += consumo_mes

    print(f"{mes:~6d} | {km_actual:>12.1f} | {km_final:>12.1f} | {consumo_mes:>18.2f}")
    km_actual = km_final

print("=" * 62)
print(f"Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
print(f"Consumo total acumulado: {consumo_acumulado:>10.2f} L")
print("=" * 62)

# -----
# MÓDULO 3: CLASIFICADOR DE CLIENTE
# -----
elif opcion == "3":
    print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

    # Validación del volumen mensual
    volumen_mensual = -1.0
    while volumen_mensual < 0:
        try:

```

```

    volumen_mensual = float(input("Ingrese el volumen de compra mensual en litros: "))
    if volumen_mensual < 0:
        print("[ERROR] El volumen de compra no puede ser un número negativo.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

es_empresa = input(";El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()

# Evaluación con if/elif/else y operadores lógicos (and, or)
if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
    categoria = "Flotilla"
    descuento = 10.0
    beneficio = "Descuento preferencial + Facturación acumulada"
elif volumen_mensual >= 100 and volumen_mensual <= 500:
    categoria = "Premium"
    descuento = 5.0
    beneficio = "Descuento en consumos + Puntos dobles"
elif volumen_mensual >= 0 and volumen_mensual < 100:
    categoria = "Regular"
    descuento = 0.0
    beneficio = "Acumulación de puntos básicos"
else:
    categoria = "No clasificado"
    descuento = 0.0
    beneficio = "Sin beneficios"

print("\n" + "=" * 52)
print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':~52}")
print("=" * 52)
print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
print(f"Registro corporativo:      {es_empresa:>12}")
print("-" * 52)
print(f"Categoría asignada:         {categoria}")
print(f"Descuento aplicable:        {descuento:>12.1f} %")
print(f"Beneficios del programa:    {beneficio}")
print("=" * 52)

# -----
# MÓDULO 4: SALIR DEL PROGRAMA
# -----
elif opcion == "4":
    print("\n" + "=" * 50)
    print(f"{'CERRANDO SISTEMA DE GESTIÓN':~50}")
    print("=" * 50)
    print(" Guardando registros de la sesión...")
    print(" Liberando memoria del sistema...")
    print(" Estado de la terminal: Finalizada correctamente.")
    print("-" * 50)
    print(" ;Gracias por utilizar el sistema de la gasolinera!")
    print(" ;Que tenga un excelente día!")
    print("=" * 50)
    break # Rompe el ciclo while True y finaliza el programa

```

PROMPT 2 *"Para el programa general, escribe el código en Python que muestre el menú y que junte los cuatro módulos, acomodalos de una forma adecuada e integra este código en un ciclo while True, si el usuario elige la opción 4, usa break, si el usuario ingresa una opción inválida, muestra un error y usa continue para reiniciar el menú. Verifica que se utilice uso correcto de int, float y str, que se use la división entera (/), el residuo*

(%) y potencia (**) en cálculos de simulación o validación. Que no se usen listas, tuplas, diccionarios, funciones personalizadas o cualquier estructura de datos avanzada (hay una tupla al inicio). Además, valida el S/N en el clasificador de cliente.”

RESPUESTA 2:

Aquí tienes la versión completa e integrada del programa. Se han eliminado todas las tuplas y estructuras de datos avanzadas (como ("1", "2", "3", "4") o ("S", "N")), se añadió la validación estricta para la opción S/N, y se integraron explícitamente los operadores numéricos requeridos (//, %, **):

Python

```
while True:
    # -----
    # DESPLIEGUE DEL MENÚ PRINCIPAL
    # -----
    print("\n" + "=" * 45)
    print("      SISTEMA DE GESTIÓN DE GASOLINERA")
    print("=" * 45)
    print("1. Venta de combustible")
    print("2. Simulación de rendimiento")
    print("3. Clasificador de cliente")
    print("4. Salir")
    print("=" * 45)

    # Captura de opción como tipo string (str)
    opcion = input("Seleccione una opción (1-4): ").strip()

    # Validación de opción sin usar listas ni tuplas
    if opcion != "1" and opcion != "2" and opcion != "3" and opcion != "4":
        print("\n[ERROR] Opción no válida. Debe ingresar un número entre 1 y 4.")
        continue # Reinicia el ciclo menú inmediatamente

    # -----
    # MÓDULO 1: VENTA DE COMBUSTIBLE
    # -----
    if opcion == "1":
        print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")
        tipo_combustible = str(input("Ingrese el tipo de combustible (ej. Magna, Premium, Diesel): "))

        # Validación de precio (float)
        precio_litro = 0.0
        while precio_litro <= 0:
            try:
                precio_litro = float(input("Ingrese el precio por litro: "))
            except ValueError:
                print("[ERROR] El precio debe ser un valor positivo mayor a cero.")

        # Validación de cantidad de litros (float)
        cantidad_litros = 0.0
        while cantidad_litros <= 0:
            try:
                cantidad_litros = float(input("Ingrese la cantidad de litros: "))
            except ValueError:
                print("[ERROR] La cantidad de litros debe ser mayor a cero.")
```

```

        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Cálculo del total a pagar (float)
total_pagar = precio_litro * cantidad_litros

# Ticket de venta formateado con f-strings, 2 decimales y alineación a la derecha
print("\n" + "=" * 40)
print("                TICKET DE VENTA                ")
print("=" * 40)
print(f"Combustible:         {tipo_combustible}")
print(f"Precio por litro: $ {precio_litro:>10.2f}")
print(f"Litros cargados:     {cantidad_litros:>10.2f} L")
print("-" * 40)
print(f"Total a pagar:       $ {total_pagar:>10.2f}")
print("=" * 40)

# -----
# MÓDULO 2: SIMULACIÓN DE RENDIMIENTO
# -----
elif opcion == "2":
    print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

    # Conversión a float de kilometraje inicial
    km_inicial = -1.0
    while km_inicial < 0:
        try:
            km_inicial = float(input("Ingrese el kilometraje inicial del vehículo (km): "))
            if km_inicial < 0:
                print("[ERROR] El kilometraje inicial no puede ser negativo.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Conversión a float del factor de rendimiento
    factor_rendimiento = 0.0
    while factor_rendimiento <= 0:
        try:
            factor_rendimiento = float(input("Ingrese el factor de rendimiento (km por litro): "))
            if factor_rendimiento <= 0:
                print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Distancia mensual (float)
    km_por_mes = 0.0
    while km_por_mes <= 0:
        try:
            km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes (km): "))
            if km_por_mes <= 0:
                print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Conversión explícita a entero (int) para la duración de la simulación
    total_meses = 0
    while total_meses <= 0:
        try:
            total_meses = int(input("Ingrese la cantidad de meses a simular: "))
            if total_meses <= 0:
                print("[ERROR] El número de meses debe ser al menos 1.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

```



```

# Impresión de la tabla de simulación
print("\n" + "=" * 62)
print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':^62}")
print("=" * 62)
print(f"{'Mes':^6} | {'Km Inicial':^12} | {'Km Final':^12} | {'Consumo Est. (L)':^18}")
print("-" * 62)

km_actual = km_inicial
consumo_acumulado = 0.0

# Ciclo for con range()
for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes

    # USO DE POTENCIA (**): Ajuste simulado por desgaste acumulado del motor
    factor_ajustado = factor_rendimiento * (0.998 ** (mes - 1))
    consumo_mes = km_por_mes / factor_ajustado
    consumo_acumulado += consumo_mes

    print(f"{'mes':^6d} | {'km_actual':>12.1f} | {'km_final':>12.1f} | {'consumo_mes':>18.2f}")
    km_actual = km_final

# USO DE DIVISIÓN ENTERA (//) Y RESIDUO (%): Cálculo del tiempo equivalente y tanques llenos
años_equivalentes = total_meses // 12
meses_sobrantes = total_meses % 12
tanques_completos = int(consumo_acumulado) // 50
litros_residuo = consumo_acumulado % 50

print("=" * 62)
print(f" Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
print(f" Consumo total acumulado:      {consumo_acumulado:>10.2f} L")
print(f" Tiempo equivalente:              {años_equivalentes} año(s) y {meses_sobrantes} mes(es)")
print(f" Estimación en tanques (50L): {tanques_completos} tanques llenos y {litros_residuo:.2f} L restantes")
print("=" * 62)

# -----
# MÓDULO 3: CLASIFICADOR DE CLIENTE
# -----
elif opcion == "3":
    print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

    # Validación de volumen mensual (float)
    volumen_mensual = -1.0
    while volumen_mensual < 0:
        try:
            volumen_mensual = float(input("Ingrese el volumen de compra mensual en litros: "))
            if volumen_mensual < 0:
                print("[ERROR] El volumen de compra no puede ser un número negativo.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Captura y VALIDACIÓN Estricta de entrada S/N (str) sin usar listas ni tuplas
    es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()
    while es_empresa != "S" and es_empresa != "N":
        print("[ERROR] Entrada no válida. Ingrese exclusivamente 'S' para Sí o 'N' para No.")
        es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()

    # Clasificación mediante if/elif/else y operadores lógicos (and, or)
    if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
        categoria = "Flotilla"

```

```

        descuento = 10.0
        beneficio = "Descuento preferencial + Facturación acumulada"
    elif volumen_mensual >= 100 and volumen_mensual <= 500:
        categoria = "Premium"
        descuento = 5.0
        beneficio = "Descuento en consumos + Puntos dobles"
    elif volumen_mensual >= 0 and volumen_mensual < 100:
        categoria = "Regular"
        descuento = 0.0
        beneficio = "Acumulación de puntos básicos"
    else:
        categoria = "No clasificado"
        descuento = 0.0
        beneficio = "Sin beneficios"

    print("\n" + "=" * 52)
    print(f'{\'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE\':~52}')
    print("=" * 52)
    print(f'Volumen mensual acumulado: {volumen_mensual:>12.2f} L')
    print(f'Registro corporativo:      {es_empresa:>12}')
    print("-" * 52)
    print(f'Categoría asignada:          {categoria}')
    print(f'Descuento aplicable:         {descuento:>12.1f} %")
    print(f'Beneficios del programa:    {beneficio}')
    print("=" * 52)

# -----
# MÓDULO 4: SALIR DEL PROGRAMA
# -----
elif opcion == "4":
    print("\n" + "=" * 50)
    print(f'{\'CERRANDO SISTEMA DE GESTIÓN\':~50}')
    print("=" * 50)
    print(" Guardando registros de la sesión...")
    print(" Liberando memoria del sistema...")
    print(" Estado de la terminal: Finalizada correctamente.")
    print("-" * 50)
    print(" ;Gracias por utilizar el sistema de la gasolinera!")
    print(" ;Que tenga un excelente día!")
    print("=" * 50)
    break # Rompe el ciclo while True limpiamente

```

Cambios realizados:

1. *Eliminación de la tupla inicial*: Se cambió `if opcion not in ("1", "2", "3", "4")`: or evaluaciones lógicas compuestas con `!=` y `and` (`if opcion != "1" and opcion != "2" and ...`), eliminando cualquier tupla del código.
2. *Validación estricta de S/N*: Se agregó un ciclo `while` dedicado en el módulo 3 (`while es_empresa != "S" and es_empresa != "N":`) que repite la solicitud hasta obtener una opción válida.
3. *Inclusión de operadores requeridos* (`**`, `//`, `%`):
 - Potencia (`**`): Utilizada en la simulación de rendimiento para calcular un factor de desgaste acumulado por mes (`0.998 ** (mes - 1)`).

- División entera (`//`): Utilizada para desglosar el total de meses simular en años completos (`total_meses // 12`) y calcular tanques llenos de 50L (`int(consumo_acumulado) // 50`).
 - Residuo (`%`): Utilizado para obtener los meses sobrantes del año (`total_meses % 12`) y el sobrante de combustible respecto al último tanque (`consumo_acumulado % 50`).
4. *Manejo estricto de tipos* (`int`, `float`, `str`): Se garantiza el casteo y la lectura explícita según el dato requerido sin invocar funciones personalizadas ni definir colecciones compuestas.

Observaciones

Usando el prompt adecuado, fue posible unir las partes en un solo programa, con ayuda de *Google Gemini*. Al ya tener gran parte estructurada la unión se dio de una forma más sencilla, mostrando la importancia de dividir el programa para codificarlo. Se observaron algunos errores después de realizar la tercera parte por primera vez, por lo que se tuvo que usar un segundo prompt para realizar correcciones. Aún así, se realizaron algunas modificaciones manualmente antes de pasar a la parte 4.

FASE 4: Pruebas de Límite y Formato

"Refina la salida visual."

A continuación se muestra la interacción con *Google Gemini* en forma de texto para la CUARTA parte, señalando el **prompt** utilizado y la **respuesta** obtenida. En el Anexo D se muestra la evidencia de la interacción en forma de capturas de pantalla.

PROMPT: *"Revisa mi código actual, muéstralo, y asegúrate de que el módulo de venta se presente con f-strings, alineación a la derecha y dos decimales; que la simulación de rendimiento use un ciclo for con range(); que el clasificador de cliente utilice if/elif/else y operadores lógicos, que los encabezados de las tablas en consola se vean profesionales y alineados, y que la finalización termine el programa de forma limpia."*

RESPUESTA:

```
Python
while True:

    # Limpieza de consola mediante secuencias ANSI
    print("\033[H\033[J", end="")

    # -----
    # DESPLIEGUE DEL MENÚ PRINCIPAL
```

```

# -----
print("\n" + "=" * 45)
print("      SISTEMA DE GESTIÓN DE GASOLINERA")
print("=" * 45)
print("1. Venta de combustible")
print("2. Simulación de rendimiento")
print("3. Clasificador de cliente")
print("4. Salir")
print("=" * 45)

# Captura de opción como tipo string (str)
opcion = input("Seleccione una opción (1-4): ").strip()

# Validación de opción sin usar listas ni tuplas
if opcion != "1" and opcion != "2" and opcion != "3" and opcion != "4":
    print("\n[ERROR] Opción no válida. Debe ingresar un número entre 1 y 4.")
    continue # Reinicia el ciclo menú inmediatamente

# -----
# MÓDULO 1: VENTA DE COMBUSTIBLE
# -----
if opcion == "1":
    print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")
    tipo_combustible = str(input("Ingrese el tipo de combustible (ej. Magna, Premium, Diesel): "))

    # Validación de precio (float)
    precio_litro = 0.0
    while precio_litro <= 0:
        try:
            precio_litro = float(input("Ingrese el precio por litro: "))
            if precio_litro <= 0:
                print("[ERROR] El precio debe ser un valor positivo mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Validación de cantidad de litros (float)
    cantidad_litros = 0.0
    while cantidad_litros <= 0:
        try:
            cantidad_litros = float(input("Ingrese la cantidad de litros: "))
            if cantidad_litros <= 0:
                print("[ERROR] La cantidad de litros debe ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Cálculo del total a pagar (float)
    total_pagar = precio_litro * cantidad_litros

    # Ticket de venta formateado con f-strings, 2 decimales y alineación a la derecha
    print("\n" + "=" * 40)
    print("          TICKET DE VENTA          ")
    print("=" * 40)
    print(f"Combustible:      {tipo_combustible}")
    print(f"Precio por litro: $ {precio_litro:>10.2f}")
    print(f"Litros cargados:  {cantidad_litros:>10.2f} L")
    print("-" * 40)
    print(f"Total a pagar:    $ {total_pagar:>10.2f}")
    print("=" * 40)

# -----
# MÓDULO 2: SIMULACIÓN DE RENDIMIENTO

```

```

# -----
elif opcion == "2":
    print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

    # Conversión a float de kilometraje inicial
    km_inicial = -1.0
    while km_inicial < 0:
        try:
            km_inicial = float(input("Ingrese el kilometraje inicial del vehículo (km): "))
            if km_inicial < 0:
                print("[ERROR] El kilometraje inicial no puede ser negativo.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Conversión a float del factor de rendimiento
    factor_rendimiento = 0.0
    while factor_rendimiento <= 0:
        try:
            factor_rendimiento = float(input("Ingrese el factor de rendimiento (km por litro): "))
            if factor_rendimiento <= 0:
                print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Distancia mensual (float)
    km_por_mes = 0.0
    while km_por_mes <= 0:
        try:
            km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes (km): "))
            if km_por_mes <= 0:
                print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Conversión explícita a entero (int) para la duración de la simulación
    total_meses = 0
    while total_meses <= 0:
        try:
            total_meses = int(input("Ingrese la cantidad de meses a simular: "))
            if total_meses <= 0:
                print("[ERROR] El número de meses debe ser al menos 1.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

    # Impresión de la tabla de simulación
    print("\n" + "=" * 62)
    print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':^62}")
    print("=" * 62)
    print(f"{'Mes':^6} | {'Km Inicial':^12} | {'Km Final':^12} | {'Consumo Est. (L)':^18}")
    print("-" * 62)

    km_actual = km_inicial
    consumo_acumulado = 0.0

    # Ciclo for con range()
    for mes in range(1, total_meses + 1):
        km_final = km_actual + km_por_mes
        consumo_mes = km_por_mes / factor_rendimiento
        consumo_acumulado += consumo_mes

    print(f"{'mes':^6d} | {'km_actual':>12.1f} | {'km_final':>12.1f} | {'consumo_mes':>18.2f}")

```

```

        km_actual = km_final

# Uso de división entera (//) y residuo (%)
años_equivalentes = total_meses // 12
meses_sobrantes = total_meses % 12
tanques_completos = int(consumo_acumulado) // 50
litros_residuo = consumo_acumulado % 50

print("=" * 62)
print(f" Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
print(f" Consumo total acumulado:      {consumo_acumulado:>10.2f} L")
print(f" Tiempo equivalente:           {años_equivalentes} año(s) y {meses_sobrantes} mes(es)")
print(f" Estimación en tanques (50L): {tanques_completos} tanques llenos y {litros_residuo:.2f} L restantes")
print("=" * 62)

# -----
# MÓDULO 3: CLASIFICADOR DE CLIENTE
# -----
elif opcion == "3":
    print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

    # Validación de volumen mensual (float)
    volumen_mensual = -1.0
    while volumen_mensual < 0:
        try:
            volumen_mensual = float(input("Ingrese el volumen de compra mensual en litros: "))
            if volumen_mensual < 0:
                print("[ERROR] El volumen de compra no puede ser un número negativo.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Captura y validación estricta de entrada S/N (str) sin usar listas ni tuplas
    es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()
    while es_empresa != "S" and es_empresa != "N":
        print("[ERROR] Entrada no válida. Ingrese exclusivamente 'S' para Sí o 'N' para No.")
        es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()

    # Clasificación mediante if/elif/else y operadores lógicos (and, or) con operador potencia (**)
    if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
        categoria = "Flotilla"
        descuento = 10.0
        beneficio = "Descuento preferencial + Facturación acumulada"
    elif volumen_mensual >= 10**2 and volumen_mensual <= 500:
        categoria = "Premium"
        descuento = 5.0
        beneficio = "Descuento en consumos + Puntos dobles"
    elif volumen_mensual >= 0 and volumen_mensual < 10**2:
        categoria = "Regular"
        descuento = 0.0
        beneficio = "Acumulación de puntos básicos"
    else:
        categoria = "No clasificado"
        descuento = 0.0
        beneficio = "Sin beneficios"

    print("\n" + "=" * 52)
    print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':~52}")
    print("=" * 52)
    print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
    print(f"Registro corporativo:      {es_empresa:>12}")
    print("-" * 52)

```

```

    print(f"Categoría asignada:      {categoria}")
    print(f"Descuento aplicable:    {descuento:>12.1f} %")
    print(f"Beneficios del programa: {beneficio}")
    print("=" * 52)

# -----
# MÓDULO 4: SALIR DEL PROGRAMA
# -----
elif opcion == "4":
    print("\n" + "=" * 50)
    print(f"'CERRANDO SISTEMA DE GESTIÓN':~50}")
    print("=" * 50)
    print(" Guardando registros de la sesión...")
    print(" Liberando memoria del sistema...")
    print(" Estado de la terminal: Finalizada correctamente.")
    print("-" * 50)
    print(" ;Gracias por utilizar el sistema de la gasolinera!")
    print(" ;Que tenga un excelente día!")
    print("=" * 50)
    break # Rompe el ciclo principal limpiamente

# Pregunta de continuidad para opciones 1, 2 y 3
while True:
    res = input("¿Deseas continuar (S/N)? ").upper()

    if res == "N":
        break
    elif res == "S":
        print()
        break
    else:
        print("..." * 50)
        print("¡Ha ocurrido un error! \nEscribiste algo distinto a 'S' o 'N'.\nIntenta nuevamente.")
        print("..." * 50)

if res == "N":
    break

print("..." * 50)
print("                ;Programa terminado!                ")
print("..." * 50)

```

Verificación de Requisitos Técnicos

- *Módulo 1 (Venta de combustible)*
 - Presentado con f-strings.
 - Formato :>10.2f aplicado para alinear montos a la derecha a 2 decimales.
- *Módulo 2 (Simulación de rendimiento):*
 - Utiliza un ciclo for iterando sobre range(1, total_meses + 1).
 - Encabezado centrado (:~6, :~12, :~18) con separadores visuales simétricos.
 - Incorpora la división entera (//) y el residuo (%) para el tiempo equivalente y desglose de tanques.

- *Módulo 3 (Clasificador de cliente):*

- Estructura selectiva `if` / `elif` / `else` combinada con los operadores `and` y `or`.
- Utiliza el operador de potencia `10**2` para marcar los umbrales de 100 L.
- Validación estricta con ciclo `while` para la opción S/N.

- *Módulo 4 (Finalización):*

- Opción 4 finaliza mediante `break` omitiendo el bloque de confirmación adicional sin errores de ejecución.
- El bloque de decisión secundaria (`res == "N"`) interrumpe el ciclo principal ordenadamente al seleccionar "N".

Observaciones

Con ayuda de *Google Gemini* se logró verificar que el programa estuviera correctamente escrito y que las salidas tuvieran una visualización limpia y profesional.

4. Reflexión

Utilizar *Google Gemini* fue de ayuda para realizar el programa. Sin embargo, se tiene que ser muy específico con los prompts que se utilizan y aún así la IA comete errores. Para identificarlos tenemos que entender lo que se tiene que hacer. Ayuda bastante entender la lógica del programa, luego dividirlo por partes para programarlo, posteriormente unirlo y finalmente revisar la estética de salida.

Anexo A: Evidencia de la FASE 1

Actúa como un experto en Python. Ayúdame a diseñar el pseudocódigo para un menú interactivo que use un ciclo while y una estructura de control para una gasolinera, sin usar funciones ni listas. El menú debe presentar cuatro opciones: 1) venta de combustible para solicitar el tipo de combustible, precio por litro y cantidad de litros; 2) simulación de rendimiento para que a partir de un kilometraje inicial se muestre el consumo estimado mes a mes o km a km en forma de tabla utilizando un ciclo for basado en un factor de rendimiento constante proporcionado por el usuario; 3) clasificador de cliente para determinar si un cliente es "Regular", "Premium" o "Flotilla" basándose en el volumen de compra mensual mediante estructuras if/elif/else y operadores lógicos; y 4) salir para terminar el programa de forma limpia.

Plaintext

```
// Inicialización de la variable de control del ciclo
opcion = 0

MIENTRAS opcion != 4 HACER:
    // -----
    // DESPLIEGUE DEL MENÚ INTERACTIVO
    // -----
    IMPRIMIR "\n=====
    IMPRIMIR " SISTEMA DE GESTIÓN DE GASOLINERA"
    IMPRIMIR "=====
    IMPRIMIR "1. Venta de combustible"
    IMPRIMIR "2. Simulación de rendimiento"
```

Plaintext

```
IMPRIMIR "1. Venta de combustible"
IMPRIMIR "2. Simulación de rendimiento"
IMPRIMIR "3. Clasificador de cliente"
IMPRIMIR "4. Salir"
IMPRIMIR "-----"

LEER opcion

// -----
// ESTRUCTURA DE CONTROL PARA LAS OPCIONES DEL MENÚ
// -----
SI opcion == 1 ENTONCES:
    IMPRIMIR "\n--- REGISTRO DE VENTA DE COMBUSTIBLE ---"
    IMPRIMIR "Ingrese tipo de combustible (Magna / Premium / Diesel):"
    LEER tipo_combustible

    IMPRIMIR "Ingrese precio por litro:"
    LEER precio_litro

    IMPRIMIR "Ingrese cantidad de litros comprados:"
    LEER cantidad_litros

    // Cálculo del costo total
    total_pagar = precio_litro * cantidad_litros

    IMPRIMIR "\n--- RESUMEN DE COMPRA ---"
    IMPRIMIR "Combustible: " + tipo_combustible
    IMPRIMIR "Litros: " + cantidad_litros + " L"
    IMPRIMIR "Total a pagar: $" + total_pagar

SINO SI opcion == 2 ENTONCES:
    IMPRIMIR "\n--- SIMULACIÓN DE RENDIMIENTO MES A MES ---"
    IMPRIMIR "Ingrese el kilometraje inicial del vehículo:"
    LEER km_inicial

    IMPRIMIR "Ingrese los kilómetros promedio recorridos por mes:"
    LEER km_por_mes

    IMPRIMIR "Ingrese el número de meses a simular:"
    LEER meses_a_simular
```

Plaintext

```

IMPRIMIR "Ingrese el número de meses a simular:"
LEER total_meses

IMPRIMIR "Ingrese el factor de rendimiento constante (km por litro):"
LEER factor_rendimiento

// Encabezado de la tabla
IMPRIMIR "\nMes | Km Inicial | Km Final | Consumo Est. (L)"
IMPRIMIR "-----"

km_actual = km_inicial

// Ciclo FOR para la proyección paso a paso
PARA mes DESDE 1 HASTA total_meses HACER:
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento

    IMPRIMIR mes + " | " + km_actual + " km | " + km_final + " km | " +

// Actualización del kilometraje acumulado para la siguiente iteración
km_actual = km_final

SINO SI opcion == 3 ENTONCES:
    IMPRIMIR "\n--- CLASIFICACIÓN DE CLIENTES ---"
    IMPRIMIR "Ingrese el volumen de compra mensual en litros:"
    LEER litros_mes

    IMPRIMIR "¿El cliente es una empresa registrada? (SI / NO):"
    LEER es_empresa

    // Clasificación mediante IF / ELIF / ELSE y operadores lógicos (AND, OR)
    SI litros_mes >= 1000 O (es_empresa == "SI" Y litros_mes >= 500) ENTONCES:
        categoria = "Flotilla"
        descuento = "10%"
    SINO SI litros_mes >= 300 Y litros_mes < 1000 ENTONCES:
        categoria = "Premium"
        descuento = "5%"
    SINO:
        categoria = "Regular"
        descuento = "0%"

    IMPRIMIR "\n--- RESULTADO DE CATEGORIZACIÓN ---"
    IMPRIMIR "Categoría asignada: " + categoria
    IMPRIMIR "Descuento aplicable: " + descuento

SINO SI opcion == 4 ENTONCES:
    IMPRIMIR "\nSaliendo del sistema... ¡Gracias por su visita!"

SINO:
    IMPRIMIR "\n[ERROR] Opción no válida. Por favor, seleccione un número entre :

```

+

@ Preguntarle a Gemini

Pensar

↻

Plaintext

```

        categoria = "Flotilla"
        descuento = "10%"
    SINO SI litros_mes >= 300 Y litros_mes < 1000 ENTONCES:
        categoria = "Premium"
        descuento = "5%"
    SINO:
        categoria = "Regular"
        descuento = "0%"

    IMPRIMIR "\n--- RESULTADO DE CATEGORIZACIÓN ---"
    IMPRIMIR "Categoría asignada: " + categoria
    IMPRIMIR "Descuento aplicable: " + descuento

SINO SI opcion == 4 ENTONCES:
    IMPRIMIR "\nSaliendo del sistema... ¡Gracias por su visita!"

SINO:
    IMPRIMIR "\n[ERROR] Opción no válida. Por favor, seleccione un número entre :

```

+

@ Preguntarle a Gemini

Pensar

↻

Elementos clave del diseño:

- **Ciclo principal `while`** : Mantiene el programa activo continuamente manipulando la variable de control `opcion` hasta que se selecciona la opción de salida (4).
- **Estructura selectiva `if / elif / else`** : Rutea el flujo hacia la tarea seleccionada y valida que el usuario introduzca una opción válida en la entrada.
- **Ciclo `for` independiente**: Utilizado en la opción 2 para iterar un rango fijo de meses calculando y formateando incrementalmente las lecturas de kilometraje y consumo en forma de tabla sin usar listas.
- **Operadores lógicos (`AND` / `OR`)**: Implementados en la opción 3 para evaluar múltiples condiciones de volumen de litros e identificar clientes comerciales para la categoría "Flotilla".

+

@ Preguntarle a Gemini

Pensar

↻

Anexo B: Evidencia de la FASE 2

Para el primer módulo, escribe el código en Python para calcular el total de una venta de combustible validando que los datos sean float positivos. Usa f-strings para mostrar el resultado con 2 decimales y el símbolo de pesos alineado.

Python

```
print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")

tipo_combustible = input("Ingrese el tipo de combustible (ej. Magna, Premium, Diesel): ")

# Validación del precio por litro (debe ser float positivo)
precio_litro = 0.0
while precio_litro <= 0:
    try:
        precio_litro = float(input("Ingrese el precio por litro: "))
        if precio_litro <= 0:
            print("[ERROR] El precio debe ser un valor positivo mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Validación de la cantidad de litros (debe ser float positivo)
cantidad_litros = 0.0
while cantidad_litros <= 0:
    try:
        cantidad_litros = float(input("Ingrese la cantidad de litros: "))
        if cantidad_litros <= 0:
            print("[ERROR] La cantidad de litros debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Cálculo del importe total
total_pagar = precio_litro * cantidad_litros
```

Python

```
# Validación de la cantidad de litros (debe ser float positivo)
cantidad_litros = 0.0
while cantidad_litros <= 0:
    try:
        cantidad_litros = float(input("Ingrese la cantidad de litros: "))
        if cantidad_litros <= 0:
            print("[ERROR] La cantidad de litros debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Cálculo del importe total
total_pagar = precio_litro * cantidad_litros

# Despliegue del ticket con alineación y 2 decimales usando f-strings
print("\n" + "=" * 40)
print("          TICKET DE VENTA          ")
print("=" * 40)
print(f"Combustible:      {tipo_combustible}")
print(f"Precio por litro:  $ {precio_litro:>10.2f}")
print(f"Litros cargados:   {cantidad_litros:>10.2f} L")
print("-" * 40)
print(f"Total a pagar:    $ {total_pagar:>10.2f}")
print("=" * 40)
```

Detalles de la implementación:

- Validación robusta: Los ciclos `while` junto con bloques `try/except ValueError` garantizan que el programa no colapse si el usuario ingresa texto y no avanzan hasta recibir un número float estrictamente mayor a 0.
- Formato de alineación: La sintaxis `>10.2f` dentro del `f-string` reserva un ancho de 10 caracteres alineados a la derecha y restringe la salida a exactamente 2 decimales, manteniendo la alineación del signo de pesos \$.

🔗 🔄 📄 ...

+ @ Preguntarle a Gemini

Pensar ✕ 0

Para el segundo módulo, escribe el código en Python para calcular el consumo estimado de un vehículo y, utilizando un ciclo for, mostrarlo en una tabla proyectada (mes a mes o km a km) del consumo estimado basado en un factor de rendimiento constante. El usuario ingresará el kilometraje inicial y el factor de rendimiento.

Python



```
print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

# Validación de Kilometraje Inicial (float >= 0)
km_inicial = -1.0
while km_inicial < 0:
    try:
        km_inicial = float(input("Ingrese el kilometraje inicial del vehiculo (km): "))
        if km_inicial < 0:
            print("[ERROR] El kilometraje inicial no puede ser negativo.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de Factor de Rendimiento en km/L (float > 0)
factor_rendimiento = 0.0
while factor_rendimiento <= 0:
    try:
        factor_rendimiento = float(input("Ingrese el factor de rendimiento (km por l): "))
        if factor_rendimiento <= 0:
            print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de kilómetros recorridos por mes (float > 0)
km_por_mes = 0.0
```

+ @ Preguntarle a Gemini

Pensar



Python



```
except ValueError:
    print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de kilómetros recorridos por mes (float > 0)
km_por_mes = 0.0
while km_por_mes <= 0:
    try:
        km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes (km): "))
        if km_por_mes <= 0:
            print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación del número de meses para el ciclo for (int > 0)
total_meses = 0
while total_meses <= 0:
    try:
        total_meses = int(input("Ingrese la cantidad de meses a simular: "))
        if total_meses <= 0:
            print("[ERROR] El número de meses debe ser al menos 1.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

# Despliegue del encabezado de la tabla formateada
print("\n" + "=" * 62)
print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':^62}")
print("=" * 62)
print(f"{'Mes':^6} | {'Km Inicial':^12} | {'Km Final':^12} | {'Consumo Est. (L)':^18}")
print("-" * 62)

km_actual = km_inicial
consumo_acumulado = 0.0

# Ciclo FOR para calcular e imprimir paso a paso sin utilizar listas
for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento
    consumo_acumulado += consumo_mes

    # Impresión con alineación y decimales usando f-strings
```

+ @ Preguntarle a Gemini

Pensar



Python

```

consumo_acumulado = 0.0

# Ciclo FOR para calcular e imprimir paso a paso sin utilizar listas
for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento
    consumo_acumulado += consumo_mes

    # Impresión con alineación y decimales usando f-strings
    print(f"mes:{^6d} | {km_actual:>12.1f} | {km_final:>12.1f} | {consumo_mes:>18.2f}")

    # Actualización de variable para la siguiente iteración
    km_actual = km_final

print("=" * 62)
print(f" Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
print(f" Consumo total acumulado: {consumo_acumulado:>10.2f} L")
print("=" * 62)

```

Detalles de la solución:

- **Generación de la tabla:** Utiliza el rango estático `range(1, total_meses + 1)` dentro del ciclo `for` para avanzar mes a mes sin necesidad de listas ni arreglos.
- **Alineación con f-strings:**
 - `:^6d` : Centra el número de mes en un espacio de 6 caracteres.
 - `>12.1f` : Alinea los kilometrajes a la derecha reservando 12 espacios con 1 decimal.
 - `>18.2f` : Alinea los litros proyectados a la derecha reservando 18 espacios con 2 decimales.
- **Control de variables acumuladoras:** Actualiza el estado de `km_actual` y `consumo_acumulado` en cada iteración manteniendo el cálculo continuo del odómetro.

👍 🗨️ 🔄 📄 ...

+ @ Preguntarle a Gemini

Pensar ✕ 0

Para el tercer módulo, escribe el código en Python para determinar si un cliente es "Regular", "Premium" o "Flotilla", basándose en el volumen de compra mensual (ej. < 100L, 100-500L, >500L), mediante estructuras `if/elif/else` y operadores lógicos.

Python

```

print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

# Validación del volumen de compra mensual (float >= 0)
volumen_mensual = -1.0
while volumen_mensual < 0:
    try:
        volumen_mensual = float(input("Ingrese el volumen de compra mensual en litros: "))
        if volumen_mensual < 0:
            print("[ERROR] El volumen de compra no puede ser un número negativo.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

# Solicitud de dato complementario para evaluar condiciones compuestas
es_empresa = input("¿El cliente cuenta con registro de empresa/Flotilla? (S/N): ").strip()

# Clasificación del cliente mediante estructuras if/elif/else y operadores lógicos
if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
    categoria = "Flotilla"
    descuento = 10.0
    beneficio = "Descuento preferencial + Facturación acumulada"
elif volumen_mensual >= 100 and volumen_mensual <= 500:
    categoria = "Premium"
    descuento = 5.0

```

+ @ Preguntarle a Gemini

Pensar ✕ 0

```

Python
# Clasificación de clientes complementando para otras condiciones complejas
es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ").strip().upper()

# Clasificación del cliente mediante estructuras if/elif/else y operadores lógicos (and, or)
if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
    categoria = "Flotilla"
    descuento = 10.0
    beneficio = "Descuento preferencial + Facturación acumulada"
elif volumen_mensual >= 100 and volumen_mensual <= 500:
    categoria = "Premium"
    descuento = 5.0
    beneficio = "Descuento en consumos + Puntos dobles"
elif volumen_mensual >= 0 and volumen_mensual < 100:
    categoria = "Regular"
    descuento = 0.0
    beneficio = "Acumulación de puntos básicos"
else:
    categoria = "No clasificado"
    descuento = 0.0
    beneficio = "Sin beneficios"

# Despliegue de resultados con formato f-string
print("\n" + "=" * 52)
print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':^52}")
print("=" * 52)
print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
print(f"Registro corporativo:      {es_empresa:>12}")
print("-" * 52)
print(f"Categoría asignada:         {categoria}")
print(f"Descuento aplicable:        {descuento:>12.1f} %")
print(f"Beneficios del programa:    {beneficio}")
print("=" * 52)

```

Detalles de la solución:

- **Uso de operadores lógicos (and , or):**
 - Para Flotilla: Aplica si el volumen es estrictamente mayor a 500L O si es una

+ @ Preguntarle a Gemini

Pensar ✕ 0

```

Python
descuento = 0.0
beneficio = "Sin beneficios"

# Despliegue de resultados con formato f-string
print("\n" + "=" * 52)
print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':^52}")
print("=" * 52)
print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
print(f"Registro corporativo:      {es_empresa:>12}")
print("-" * 52)
print(f"Categoría asignada:         {categoria}")
print(f"Descuento aplicable:        {descuento:>12.1f} %")
print(f"Beneficios del programa:    {beneficio}")
print("=" * 52)

```

Detalles de la solución:

- **Uso de operadores lógicos (and , or):**
 - Para Flotilla: Aplica si el volumen es estrictamente mayor a 500L O si es una empresa con compras desde 300L (`volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300)`).
 - Para Premium: Evalúa que el volumen esté dentro del rango de 100L a 500L inclusive utilizando el operador `and` (`volumen_mensual >= 100 and volumen_mensual <= 500`).
 - Para Regular: Define compras estrictamente menores a 100L (`volumen_mensual >= 0 and volumen_mensual < 100`).
- **Tratamiento de cadenas:** Limpia espacios con `.strip()` y convierte a mayúsculas con `.upper()` en la entrada del usuario para evitar errores de captura en la respuesta 'S' / 'N'.

🔗 🗨️ 🔄 📄 ...

+ @ Preguntarle a Gemini

Pensar ✕ 0

Para el cuarto módulo, escribe el código en Python para que termine el programa de forma limpia

Python



```
print("\n" + "=" * 50)
print(f"¡CERRANDO SISTEMA DE GESTIÓN!^50}")
print("=" * 50)
print(" Guardando registros de la sesión...")
print(" Liberando memoria del sistema...")
print(" Estado de la terminal: Finalizada correctamente.")
print("-" * 50)
print(" ¡Gracias por utilizar el sistema de la gasolinera!")
print(" ¡Que tenga un excelente día!")
print("=" * 50)

# Cambio de la variable de control del ciclo while principal
# (Al cambiar el valor de la opción a 4 o 'ejecutando' a False,
# el ciclo finaliza de forma limpia sin romper abruptamente el programa)
opcion = 4
```

Detalles de la solución:

- **Terminación estructurada:** A diferencia del uso de comandos de interrupción forzada como `sys.exit()`, el cierre se realiza mediante la modificación de la variable de control del ciclo `while principal (opcion = 4)`. Esto permite que el flujo del programa complete su iteración de forma natural y limpia.
- **Confirmación visual:** Incluye un mensaje explícito de cierre que le da certeza al usuario de que las operaciones finalizaron correctamente antes de devolver el control a la terminal.

A B C D

+ @ Preguntarle a Gemini

Pensar × 0

Anexo C: Evidencia de la FASE 3

Para el programa general, escribe el código en Python que muestre el menú y que junte los cuatro módulos, acomodalos de una forma adecuada e integra este código en un ciclo while True, si el usuario elige la opción 4, usa break, si el usuario ingresa una opción inválida, muestra un error y usa continue para reiniciar el menú.

Python

```
while True:
    # -----
    # DESPLIEGUE DEL MENÚ PRINCIPAL
    # -----
    print("\n" + "=" * 45)
    print("      SISTEMA DE GESTIÓN DE GASOLINERA")
    print("=" * 45)
    print("1. Venta de combustible")
    print("2. Simulación de rendimiento")
    print("3. Clasificador de cliente")
    print("4. Salir")
    print("=" * 45)

    opcion = input("Seleccione una opción (1-4): ").strip()

    # Validación de la opción seleccionada
    if opcion not in ("1", "2", "3", "4"):
        print("\n[ERROR] Opción no válida. Debe ingresar un número entre 1 y 4.")
        continue # Reinicia el ciclo while inmediatamente

    # -----
    # MÓDULO 1: VENTA DE COMBUSTIBLE
    # -----
    if opcion == "1":
        print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")
        tipo_combustible = input("Ingrese el tipo de combustible (ej. Magna, Premium) ")

        # Validación del precio por litro
        precio_litro = 0.0
        while precio_litro <= 0:
            try:
                precio_litro = float(input("Ingrese el precio por litro: "))
                if precio_litro <= 0:
                    print("[ERROR] El precio debe ser un valor positivo mayor a cero")
```

+ Preguntarle a Gemini

Pensar ▾



Python

```

# Validación del precio por litro
precio_litro = 0.0
while precio_litro <= 0:
    try:
        precio_litro = float(input("Ingrese el precio por litro: "))
        if precio_litro <= 0:
            print("[ERROR] El precio debe ser un valor positivo mayor a cero")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal")

# Validación de la cantidad de litros
cantidad_litros = 0.0
while cantidad_litros <= 0:
    try:
        cantidad_litros = float(input("Ingrese la cantidad de litros: "))
        if cantidad_litros <= 0:
            print("[ERROR] La cantidad de litros debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal")

# Cálculo del importe total
total_pagar = precio_litro * cantidad_litros

# Ticket de venta
print("\n" + "=" * 40)
print("          TICKET DE VENTA          ")
print("=" * 40)
print(f"Combustible:      {tipo_combustible}")
print(f"Precio por litro: $ {precio_litro:>10.2f}")
print(f"Litros cargados:   {cantidad_litros:>10.2f} L")
print("-" * 40)
print(f"Total a pagar:     $ {total_pagar:>10.2f}")
print("=" * 40)

# -----
# MÓDULO 2: SIMULACIÓN DE RENDIMIENTO
# -----
elif opcion == "2":
    print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

    # Validación de Kilometraje Inicial
    km_inicial = -1.0
    while km_inicial < 0:
        try:
            km_inicial = float(input("Ingrese el kilometraje inicial del vehículo: "))
            if km_inicial < 0:
                print("[ERROR] El kilometraje inicial no puede ser negativo.")
        except ValueError:
            print("[ERROR] Entrada inválida. Ingrese un número entero o decimal")

```

+ Preguntarle a Gemini

Pensar ▾



Python

```

except ValueError:
    print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de Factor de Rendimiento
factor_rendimiento = 0.0
while factor_rendimiento <= 0:
    try:
        factor_rendimiento = float(input("Ingrese el factor de rendimiento (0.0 a 1.0): "))
        if factor_rendimiento <= 0:
            print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación de kilómetros mensuales
km_por_mes = 0.0
while km_por_mes <= 0:
    try:
        km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes (km): "))
        if km_por_mes <= 0:
            print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
    except ValueError:
        print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

# Validación del número de meses
total_meses = 0
while total_meses <= 0:
    try:
        total_meses = int(input("Ingrese la cantidad de meses a simular: "))
        if total_meses <= 0:
            print("[ERROR] El número de meses debe ser al menos 1.")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

# Despliegue de la tabla proyectada
print("\n" + "=" * 62)
print(f'{ "TABLA PROYECTADA DE CONSUMO MENSUAL":^62}')
print("=" * 62)
print(f'{ "Mes":^6} | { "Km Inicial":^12} | { "Km Final":^12} | { "Consumo Est.":^12}')
print("-" * 62)

km_actual = km_inicial
consumo_acumulado = 0.0

for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento
    consumo_acumulado += consumo_mes

    print(f'{mes:^6d} | {km_actual:>12.1f} | {km_final:>12.1f} | {consumo_mes:>12.1f}')

```

+ Preguntarle a Gemini

Pensar ▾



Python

```

        print(f"{mes:^6d} | {km_actual:>12.1f} | {km_final:>12.1f} | {consumo_mes:  

        km_actual = km_final

    print("=" * 62)
    print(f" Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
    print(f" Consumo total acumulado:      {consumo_acumulado:>10.2f} L")
    print("=" * 62)

# -----
# MÓDULO 3: CLASIFICADOR DE CLIENTE
# -----
elif opcion == "3":
    print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

    # Validación del volumen mensual
    volumen_mensual = -1.0
    while volumen_mensual < 0:
        try:
            volumen_mensual = float(input("Ingrese el volumen de compra mensual o  

            if volumen_mensual < 0:
                print("[ERROR] El volumen de compra no puede ser un número negati
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o dec

    es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N)

    # Evaluación con if/elif/else y operadores lógicos (and, or)
    if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
        categoria = "Flotilla"
        descuento = 10.0
        beneficio = "Descuento preferencial + Facturación acumulada"
    elif volumen_mensual >= 100 and volumen_mensual <= 500:
        categoria = "Premium"
        descuento = 5.0
        beneficio = "Descuento en consumos + Puntos dobles"
    elif volumen_mensual >= 0 and volumen_mensual < 100:
        categoria = "Regular"
        descuento = 0.0
        beneficio = "Acumulación de puntos básicos"
    else:
        categoria = "No clasificado"
        descuento = 0.0
        beneficio = "Sin beneficios"

    print("\n" + "=" * 52)
    print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':^52}")
    print("=" * 52)
    print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
    print(f"Registro corporativo:      {es_empresa:>12}")

```

+ Preguntarle a Gemini

Pensar ▾



Python

```
# Evaluación con if/elif/else y operadores lógicos (and, or)
if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
    categoria = "Flotilla"
    descuento = 10.0
    beneficio = "Descuento preferencial + Facturación acumulada"
elif volumen_mensual >= 100 and volumen_mensual <= 500:
    categoria = "Premium"
    descuento = 5.0
    beneficio = "Descuento en consumos + Puntos dobles"
elif volumen_mensual >= 0 and volumen_mensual < 100:
    categoria = "Regular"
    descuento = 0.0
    beneficio = "Acumulación de puntos básicos"
else:
    categoria = "No clasificado"
    descuento = 0.0
    beneficio = "Sin beneficios"

print("\n" + "=" * 52)
print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':^52}")
print("=" * 52)
print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
print(f"Registro corporativo:      {es_empresa:>12}")
print("-" * 52)
print(f"Categoría asignada:          {categoria}")
print(f"Descuento aplicable:         {descuento:>12.1f} %")
print(f"Beneficios del programa:     {beneficio}")
print("=" * 52)

# -----
# MÓDULO 4: SALIR DEL PROGRAMA
# -----
elif opcion == "4":
    print("\n" + "=" * 50)
    print(f"{'CERRANDO SISTEMA DE GESTIÓN':^50}")
    print("=" * 50)
    print(" Guardando registros de la sesión...")
    print(" Liberando memoria del sistema...")
    print(" Estado de la terminal: Finalizada correctamente.")
    print("-" * 50)
    print(" ¡Gracias por utilizar el sistema de la gasolinera!")
    print(" ¡Que tenga un excelente día!")
    print("=" * 50)
    break # Rompe el ciclo while True y finaliza el programa
```

    ...  Preguntarle a Gemini

Pensar ▾



Para el programa general, escribe el código en Python que muestre el menú y que junte los cuatro módulos, acomódalos de una forma adecuada e integra este código en un ciclo while True, si el usuario elige la opción 4, usa break, si el usuario ingresa una opción inválida, muestra un error y usa continue para reiniciar el menú. Verifica que se utilice uso correcto de int, float y str, que se use la división entera (//), el residuo (%) y potencia (**) en cálculos de simulación o validación. Que no se usen listas, tuplas, diccionarios, funciones personalizadas o cualquier estructura de datos avanzada (hay una dupla al inicio). Además, valida el S/N en el clasificador de cliente.

Aquí tienes la versión completa e integrada del programa. Se han eliminado todas las tuplas y estructuras de datos avanzadas (como ("1", "2", "3", "4") o ("S", "N")), se añadió la validación estricta para la opción S/N, y se integraron explícitamente los operadores numéricos requeridos (// , % , **):

Python

```
while True:
# -----
# DESPLIEGUE DEL MENÚ PRINCIPAL
# -----
print("\n" + "=" * 45)
print("      SISTEMA DE GESTIÓN DE GASOLINERA")
print("=" * 45)
print("1. Venta de combustible")
print("2. Simulación de rendimiento")
print("3. Clasificador de cliente")
print("4. Salir")
print("=" * 45)

# Captura de opción como tipo string (str)
opcion = input("Seleccione una opción (1-4): ").strip()

# Validación de opción sin usar listas ni tuplas
if opcion != "1" and opcion != "2" and opcion != "3" and opcion != "4":
    print("\n[ERROR] Opción no válida. Debe ingresar un número entre 1 y 4.")
    continue # Reinicia el ciclo menú inmediatamente
```

+ @ Preguntarle a Gemini

Pensar

```
# -----
# MÓDULO 4: SALIR DEL PROGRAMA
# -----
elif opcion == "4":
    print("\n" + "=" * 50)
    print(f"¡CERRANDO SISTEMA DE GESTIÓN!:{50}")
    print("=" * 50)
    print(" Guardando registros de la sesión...")
    print(" Liberando memoria del sistema...")
    print(" Estado de la terminal: Finalizada correctamente.")
    print("=" * 50)
    print(" ¡Gracias por utilizar el sistema de la gasolinera!")
    print(" ¡Que tenga un excelente día!")
    print("=" * 50)
    break # Rompe el ciclo while True limpiamente
```

Cambios realizados:

1. Eliminación de la tupla inicial: Se cambió `if opcion not in ("1", "2", "3", "4")` por evaluaciones lógicas compuestas con `!=` y `and` (`if opcion != "1" and opcion != "2" and ...`), eliminando cualquier tupla del código.
2. Validación estricta de S/N: Se agregó un ciclo `while` dedicado en el módulo 3 (`while es_empresa != "S" and es_empresa != "N":`) que repite la solicitud hasta obtener una opción válida.
3. Inclusión de operadores requeridos (`**`, `//`, `%`):
 - o Potencia (`**`): Utilizada en la simulación de rendimiento para calcular un factor de desgaste acumulado por mes (`0.998 ** (mes - 1)`).
 - o División entera (`//`): Utilizada para desglosar el total de meses simular en años completos (`total_meses // 12`) y calcular tanques llenos de 50L (`int(consumo_acumulado // 50)`).
 - o Residuo (`%`): Utilizado para obtener los meses sobrantes del año (`total_meses % 12`) y el sobrante de combustible respecto al último tanque (`consumo_acumulado % 50`).
4. Manejo estricto de tipos (`int`, `float`, `str`): Se garantiza el casteo y la lectura explícita según el dato requerido sin invocar funciones personalizadas ni definir colecciones compuestas.

👍 🗑️ 🔄 📄 ...

+ @ Preguntarle a Gemini

Pensar

Anexo D: Evidencia de la FASE 4

Revisa mi código actual, muéstralo, y asegúrate de que el módulo de venta se presente con f-strings, alineación a la derecha y dos decimales; que la simulación de rendimiento use un ciclo for con range(); que el clasificador de cliente utilice if/elif/else y operadores lógicos, que los encabezados de las tablas en consola se vean profesionales y alineados, y que la finalización termine el programa de forma limpia.

Python

```
while True:

    # Limpieza de consola mediante secuencias ANSI
    print("\033[H\033[J", end="")

    # -----
    # DESPLIEGUE DEL MENÚ PRINCIPAL
    # -----

    print("\n" + "=" * 45)
    print("      SISTEMA DE GESTIÓN DE GASOLINERA")
    print("=" * 45)
    print("1. Venta de combustible")
    print("2. Simulación de rendimiento")
    print("3. Clasificador de cliente")
    print("4. Salir")
    print("=" * 45)

    # Captura de opción como tipo string (str)
    opcion = input("Seleccione una opción (1-4): ").strip()

    # Validación de opción sin usar listas ni tuplas
    if opcion != "1" and opcion != "2" and opcion != "3" and opcion != "4":
        print("\n[ERROR] Opción no válida. Debe ingresar un número entre 1 y 4.")
        continue # Reinicia el ciclo menú inmediatamente

    # -----
    # MÓDULO 1: VENTA DE COMBUSTIBLE
    # -----

    if opcion == "1":
        print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")
        tipo_combustible = str(input("Ingrese el tipo de combustible (ej. Magna, Pre
```

Python

```
# MÓDULO 1: VENTA DE COMBUSTIBLE
# -----

if opcion == "1":
    print("\n--- MÓDULO 1: VENTA DE COMBUSTIBLE ---")
    tipo_combustible = str(input("Ingrese el tipo de combustible (ej. Magna, Pre

    # Validación de precio (float)
    precio_litro = 0.0
    while precio_litro <= 0:
        try:
            precio_litro = float(input("Ingrese el precio por litro: "))
            if precio_litro <= 0:
                print("[ERROR] El precio debe ser un valor positivo mayor a cero")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o de

    # Validación de cantidad de litros (float)
    cantidad_litros = 0.0
    while cantidad_litros <= 0:
        try:
            cantidad_litros = float(input("Ingrese la cantidad de litros: "))
            if cantidad_litros <= 0:
                print("[ERROR] La cantidad de litros debe ser mayor a cero.")
        except ValueError:
            print("[ERROR] Entrada inválida. Debe ingresar un número entero o de

    # Cálculo del total a pagar (float)
    total_pagar = precio_litro * cantidad_litros

    # Ticket de venta formateado con f-strings, 2 decimales y alineación a la de
    print("\n" + "=" * 40)
    print("      TICKET DE VENTA      ")
    print("=" * 40)
    print(f"Combustible:      {tipo_combustible}")
    print(f"Precio por litro: $ {precio_litro:>10.2f}")
    print(f"Litros cargados:      {cantidad_litros:>10.2f} L")
    print("-" * 40)
    print(f"Total a pagar:      $ {total_pagar:>10.2f}")
    print("=" * 40)

    # -----
    # MÓDULO 2: SIMULACIÓN DE RENDIMIENTO
    # -----

    elif opcion == "2":
        print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

        # Conversión a float de kilometraje inicial
        km_inicial = -1.0
        while km_inicial < 0:
```

```

Python
elif opcion == "2":
    print("\n--- MÓDULO 2: SIMULACIÓN DE RENDIMIENTO Y CONSUMO ---")

    # Conversión a float de kilometraje inicial
    km_inicial = -1.0
    while km_inicial < 0:
        try:
            km_inicial = float(input("Ingrese el kilometraje inicial del vehículo: "))
            if km_inicial < 0:
                print("[ERROR] El kilometraje inicial no puede ser negativo.")
            except ValueError:
                print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Conversión a float del factor de rendimiento
    factor_rendimiento = 0.0
    while factor_rendimiento <= 0:
        try:
            factor_rendimiento = float(input("Ingrese el factor de rendimiento (km/l): "))
            if factor_rendimiento <= 0:
                print("[ERROR] El factor de rendimiento debe ser mayor a cero.")
            except ValueError:
                print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Distancia mensual (float)
    km_por_mes = 0.0
    while km_por_mes <= 0:
        try:
            km_por_mes = float(input("Ingrese la distancia estimada a recorrer por mes: "))
            if km_por_mes <= 0:
                print("[ERROR] Los kilómetros mensuales deben ser mayor a cero.")
            except ValueError:
                print("[ERROR] Entrada inválida. Ingrese un número entero o decimal.")

    # Conversión explícita a entero (int) para la duración de la simulación
    total_meses = 0
    while total_meses <= 0:
        try:
            total_meses = int(input("Ingrese la cantidad de meses a simular: "))
            if total_meses <= 0:
                print("[ERROR] El número de meses debe ser al menos 1.")
            except ValueError:
                print("[ERROR] Entrada inválida. Debe ingresar un número entero.")

    # Impresión de la tabla de simulación
    print("\n" + "=" * 62)
    print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':^62}")
    print("=" * 62)
    print(f"{'Mes':^6} | {'Km Inicial':^12} | {'Km Final':^12} | {'Consumo Est.':^12}")
    print("-" * 62)

```

```

+ @ Preguntarle a Gemini
Pensar
Python
# Impresión de la tabla de simulación
print("\n" + "=" * 62)
print(f"{'TABLA PROYECTADA DE CONSUMO MENSUAL':^62}")
print("=" * 62)
print(f"{'Mes':^6} | {'Km Inicial':^12} | {'Km Final':^12} | {'Consumo Est.':^12}")
print("-" * 62)

km_actual = km_inicial
consumo_acumulado = 0.0

# Ciclo for con range()
for mes in range(1, total_meses + 1):
    km_final = km_actual + km_por_mes
    consumo_mes = km_por_mes / factor_rendimiento
    consumo_acumulado += consumo_mes

    print(f"{'Mes':^6d} | {'km_actual':>12.1f} | {'km_final':>12.1f} | {'consumo_mes':>12.1f}")
    km_actual = km_final

# Uso de división entera (//) y residuo (%)
años_equivalentes = total_meses // 12
meses_sobrantes = total_meses % 12
tanques_completos = int(consumo_acumulado) // 50
litros_residuo = consumo_acumulado % 50

print("\n" + "=" * 62)
print(f"Distancia total proyectada: {total_meses * km_por_mes:>10.1f} km")
print(f"Consumo total acumulado: {consumo_acumulado:>10.2f} L")
print(f"Tiempo equivalente: {años_equivalentes} año(s) y {meses_sobrantes} meses")
print(f"Estimación en tanques (50L): {tanques_completos} tanques llenos y {litros_residuo} L")
print("=" * 62)

# MÓDULO 3: CLASIFICADOR DE CLIENTE
# -----
elif opcion == "3":
    print("\n--- MÓDULO 3: CLASIFICACIÓN DE CLIENTES ---")

    # Validación de volumen mensual (float)
    volumen_mensual = -1.0
    while volumen_mensual < 0:
        try:
            volumen_mensual = float(input("Ingrese el volumen de compra mensual: "))
            if volumen_mensual < 0:
                print("[ERROR] El volumen de compra no puede ser un número negativo.")
            except ValueError:
                print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal.")

    # Captura y validación estricta de entrada S/N (str) sin usar listas ni tuplas

```

```

+ @ Preguntarle a Gemini
Pensar

```

```

Python
    if volumen_mensual < 0:
        print("[ERROR] El volumen de compra no puede ser un número negativo")
    except ValueError:
        print("[ERROR] Entrada inválida. Debe ingresar un número entero o decimal")

    # Captura y validación estricta de entrada S/N (str) sin usar listas ni tuplas
    es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? (S/N): ")
    while es_empresa != "S" and es_empresa != "N":
        print("[ERROR] Entrada no válida. Ingrese exclusivamente 'S' para Sí o 'N' para No.")
    es_empresa = input("¿El cliente cuenta con registro de empresa/flotilla? ")

    # Clasificación mediante if/elif/else y operadores lógicos (and, or) con operadores de comparación
    if volumen_mensual > 500 or (es_empresa == "S" and volumen_mensual >= 300):
        categoria = "Flotilla"
        descuento = 10.0
        beneficio = "Descuento preferencial + Facturación acumulada"
    elif volumen_mensual >= 10**2 and volumen_mensual <= 500:
        categoria = "Premium"
        descuento = 5.0
        beneficio = "Descuento en consumos + Puntos dobles"
    elif volumen_mensual >= 0 and volumen_mensual < 10**2:
        categoria = "Regular"
        descuento = 0.0
        beneficio = "Acumulación de puntos básicos"
    else:
        categoria = "No clasificado"
        descuento = 0.0
        beneficio = "Sin beneficios"

    print("\n" + "=" * 52)
    print(f"{'DIAGNÓSTICO DE CATEGORÍA DE CLIENTE':^52}")
    print("=" * 52)
    print(f"Volumen mensual acumulado: {volumen_mensual:>12.2f} L")
    print(f"Registro corporativo: {es_empresa:>12}")
    print("-" * 52)
    print(f"Categoría asignada: {categoria}")
    print(f"Descuento aplicable: {descuento:>12.1f} %")
    print(f"Beneficios del programa: {beneficio}")
    print("-" * 52)

    #
    # MÓDULO 4: SALIR DEL PROGRAMA
    #
    elif opcion == "4":
        print("\n" + "=" * 50)
        print(f"{'CERRANDO SISTEMA DE GESTIÓN':^50}")
        print("=" * 50)
        print("Guardando registros de la sesión...")
        print("Liberando memoria del sistema...")

        print("Descuento aplicable: {descuento:>12.1f} %")
        print("Beneficios del programa: {beneficio}")
        print("-" * 52)

        #
        # MÓDULO 4: SALIR DEL PROGRAMA
        #
        elif opcion == "4":
            print("\n" + "=" * 50)
            print(f"{'CERRANDO SISTEMA DE GESTIÓN':^50}")
            print("=" * 50)
            print("Guardando registros de la sesión...")
            print("Liberando memoria del sistema...")
            print("Estado de la terminal: Finalizada correctamente.")
            print("-" * 50)
            print("¡Gracias por utilizar el sistema de la gasolinera!")
            print("¡Que tenga un excelente día!")
            print("-" * 50)
            break # Rompe el ciclo principal limpiamente

        # Pregunta de continuidad para opciones 1, 2 y 3
        while True:
            res = input("¿Deseas continuar (S/N)? ").upper()

            if res == "N":
                break
            elif res == "S":
                print()
                break
            else:
                print("-" * 50)
                print("❌ ¡Ha ocurrido un error! ❌ \nEscribiste algo distinto a 'S' o 'N'.")
                print("-" * 50)

        if res == "N":
            break

        print("-" * 50)
        print("■ ¡Programa terminado! ■")
        print("-" * 50)

```

Verificación de Requisitos Técnicos

- Módulo 1 (Venta de combustible):
 - Presentado con f-strings.


```
Python
# ...
break
else:
    print("." * 50)
    print("❌ ¡Ha ocurrido un error! ❌ \nEscribiste algo distinto a 'S' o 'N'")
    print("." * 50)

if res == "N":
    break

print("." * 50)
print("✅ ¡Programa terminado! ✅")
print("." * 50)
```

Verificación de Requisitos Técnicos

- **Módulo 1 (Venta de combustible):**
 - Presentado con f-strings.
 - Formato `>10.2f` aplicado para alinear montos a la derecha a 2 decimales.
- **Módulo 2 (Simulación de rendimiento):**
 - Utiliza un ciclo `for` iterando sobre `range(1, total_meses + 1)`.
 - Encabezado centrado (`^6` , `^12` , `^18`) con separadores visuales simétricos.
 - Incorpora la división entera (`//`) y el residuo (`%`) para el tiempo equivalente y desglose de tanques.
- **Módulo 3 (Clasificador de cliente):**
 - Estructura selectiva `if / elif / else` combinada con los operadores `and` y `or`.
 - Utiliza el operador de potencia `10**2` para marcar los umbrales de 100 L.
 - Validación estricta con ciclo `while` para la opción S/N.
- **Módulo 4 y Finalización:**
 - Opción 4 finaliza mediante `break` omitiendo el bloque de confirmación adicional sin errores de ejecución.
 - El bloque de decisión secundaria (`res == "N"`) interrumpe el ciclo principal ordenadamente al seleccionar "N".

+ Preguntarle a Gemini Pensar